Programming
Human-Centered Artificial Intelligence
Dr. Ramon Winterhalder
A.Y. 2026/27

Exercise sheet 5 — Solutions

Strings – text as a sequence

Published after the in-class discussion. Read these only once you have tried the sheet.

Part A — Strings as sequences

A1 *Twelve slices*

`w = "Programming"`, eleven characters, indices 0 to 10.

	slice	result
1	`w[0]`	`"P"`
2	`w[4]`	`"r"` — the second r, at index 4
3	`w[-1]`	`"g"`
4	`w[-4]`	`"m"`
5	`w[0:3]`	`"Pro"`
6	`w[3:7]`	`"gram"`
7	`w[:5]`	`"Progr"`
8	`w[7:]`	`"ming"`
9	`w[-3:]`	`"ing"`
10	`w[::3]`	`"Pgmn"` — indices 0, 3, 6, 9
11	`w[::-1]`	`"gnimmargorP"`
12	`w[4:4]`	`""` — the empty string

The two worth checking twice are **2** and **10**. Number 2 catches anyone who counted from 1. Number 10 catches anyone who assumed a step of 3 gives three characters — it gives every third one, and there are four of those.

Number 12 asks for everything from index 4 up to but not including index 4, which is nothing at all. This is not an error: an empty string is a perfectly good string, with `len(w[4:4]) == 0`.

Worth noticing. Slicing never raises `IndexError`, even when the numbers are nonsense: `w[50:90]` is `""` and `w[5:2]` is `""`. Only single-character indexing complains. That is convenient and occasionally dangerous — a slice that silently returns nothing is a bug that does not announce itself.

A2 *True or False*

1	`True`	`"gram"` appears at index 3
2	`False`	`w[0]` is `"P"`, capital. Comparison is case-sensitive
3	`True`	eleven characters
4	`True`	see below
5	`True`	same first two characters, then `"c"` before `"d"`
6	`True`	index −1 and index 10 are the same character

Number 4. Strings are compared character by character using each character's numeric code, and in that ordering every capital letter comes before every lower-case one. So `"Z"` (90) is less than `"a"` (97), and `"Zoo" < "apple"` is `True`.

That is almost never what a human means by alphabetical order.

Worth noticing. This is why a sorted list of names puts `Zoe` before `ada`. The standard fix is to compare `s.lower()` on both sides — which is also the answer to sheet 3, B1 number 6, where we promised this explanation.

A3 *Write the slice*

	slice	
1	`w[:3]`	`"Pro"`
2	`w[-3:]`	`"ing"` — negative index, no `len()` needed
3	`w[1:]`	`"rogramming"`
4	`w[:-2]`	`"Programmi"` — a negative *stop* also works
5	`w[5]`	`"a"` — eleven characters, so index 5 has five either side
6	`w[::-1]`	`"gnimmargorP"`

Number 4 is the one most people write as `w[0:len(w)-2]`. It works, and `w[:-2]` says the same thing in four characters. Negative numbers work on both ends of a slice, which is exactly why you rarely need `len()` inside one.

A4 *Why does this fail?*

a) `TypeError: 'str' object does not support item assignment.`

b) You can *read* a character out of a string with `word[0]`, but you cannot *write* one back in. Strings are immutable.

c)

```
word = "cat"
word = "b" + word[1:]
print(word)          # bat
```

d) No. The string `"cat"` was never modified — a *new* string `"bat"` was built, and the name `word` was re-attached to it. Nothing points at `"cat"` any more, which is a different thing from it having changed.

Worth noticing. Immutability is not a restriction Python imposes to annoy you. It is what lets you pass a string to any part of a program without worrying that something will alter it behind your back. Lists, next week, are mutable — and that one difference is the source of every confusing bug in lecture 6.

A5 *Count something*

```
# ask for a line of text
# a counter, starting at 0
# for each character:
#     if it is a digit, count it
# print the count

text = input("Text: ")

digits = 0

for ch in text:
    if ch in "0123456789":
        digits = digits + 1

print(f"{digits} digits")
```

`a1b2c3` gives 3; a line with no digits gives 0, because the accumulator's starting value is already the right answer for "none found".

It is the **accumulator** from lecture 4, walking over characters instead of numbers. Third context, same shape.

Worth noticing. `ch in "0123456789"` is worth a second look. `in` asks whether one string appears inside another, and a single character is a string, so membership in a ten-character string is a compact way to say "is it a digit". The same trick gives you `ch in "aeiou"` for vowels.

Part B — String methods

B1 *What comes back?*

```
t = " Ada Lovelace "
```

	call	result
1	`t.strip()`	`"Ada Lovelace"` — ends only; the middle pair stays
2	`t.lower()`	`" ada lovelace "`
3	`t.upper()`	`" ADA LOVELACE "`
4	`t.replace(" ", "")`	`"AdaLovelace"` — every space, including the middle
5	`t.count("a")`	`2`
6	`t.find("Lov")`	`7`
7	`t.strip().startswith("Ada")`	`True`
8	`t.split()`	`["Ada", "Lovelace"]`

Number 5 is the trap. `"Ada Lovelace"` contains three letter-a's, but only **two** lower-case ones. `.count("a")` is case-sensitive like everything else, and the capital `A` does not match. `t.lower().count("a")` gives 3.

Number 8 gives back a **list** — the square brackets are the clue. That is next week's subject. For now: you can loop over it and ask for its `len()`.

Worth noticing. Notice the difference between `.strip()` and `.replace(" ", "")`. `strip` removes whitespace only at the two ends; `replace` removes every occurrence anywhere. Choosing the wrong one is how names end up looking like `AdaLovelace`.

B2 *These do nothing*

a) Number 1 prints `ada`. Number 2 prints `[  hi  ]`.

b) They were meant to print `ADA` and `[hi]`.

c) Catch what the method gives back:

```
name = name.upper()
message = message.strip()
```

In one sentence: **a string method never changes the string — it returns a new one, and if you do not assign it to something, it is thrown away.**

Worth noticing. This is the single most common string bug, and it is completely silent. No error, no warning, and the program carries on with the old value. Whenever a string operation "does not work", check first whether you caught the result.

B3 *It runs, and the count is wrong*

a) For `Ada Lovelace` it gives **5**.

b) It should give **6**.

c) The capital `A` at the start. The test is `ch in "aeiou"`, and `"A"` is not in that string — only the lower-case vowels are.

d) One line:

```
for ch in name.lower():
```

Adding the capitals to the test string (`ch in "aeiouAEIOU"`) also works and is worse: it is longer, easier to mistype, and it does not help with the next thing you compare.

Worth noticing. Three questions on this sheet — A2 number 2, B1 number 5 and this one — are all the same fact from different directions: *everything* about strings is case-sensitive. The habit worth building is to normalise with `.lower()` once, early, and then stop thinking about it.

B4 *Clean up a name*

```
raw = "  aDA   loveLACE  "

clean = " ".join(raw.split()).title()

print(clean)          # Ada Lovelace
```

Read it from the inside out:

a) `raw.split()` gives `["aDA", "loveLACE"]` — splitting on whitespace throws away runs of spaces and the ones at the ends, all at once;

b) `" ".join(...)` puts them back together with exactly one space;

c) `.title()` capitalises the first letter of each word and lowers the rest.

`raw.strip().title()` is a common near-miss: it fixes the ends and the capitals but leaves the three spaces in the middle.

Worth noticing. `.title()` is convenient and naive: it turns `"o'brien"` into `"O'Brien"` (right) and `"mcdonald"` into `"Mcdonald"` (wrong). Real name handling is genuinely hard, and any rule you can write in one line will be wrong for somebody. Worth knowing before you write a form that does it.

B5 *Initials*

```
full = input("Full name: ")

initials = ""

for word in full.split():
    initials = initials + word[0].upper() + "."

print(initials)
```

`ada byron lovelace` gives `A.B.L.`; `grace hopper` gives `G.H.`

It works for any number of names because the loop asks the text how many there are rather than being told. Anyone who wrote three separate lines has a program that is correct for exactly one input.

The accumulator again — building a string this time, as in the palindrome example.

B6 *Is it a CSV file?*

```
name = input("File name: ")

if name.lower().endswith(".csv"):
    print("yes")
else:
    print("no")
```

input	answer	
`data.csv`	yes	
`data.CSV`	yes	**this is the one that breaks a first attempt**
`data.csv.txt`	no	it ends in `.txt`; `.csv` in the middle does not count
`csv`	no	no dot, so it is not an extension

The fix: `.lower()` before the test, because file extensions are not case-sensitive in practice and users type whatever they like.

Note why `".csv" in name` would be the wrong test — it says *yes* to `data.csv.txt`. `endswith` asks the question you actually mean.

Worth noticing. Three of the four inputs exist to attack a different wrong answer: `data.CSV` attacks forgetting case, `data.csv.txt` attacks using `in`, and `csv` attacks testing for `".csv"` without the dot. Choosing test cases that attack specific mistakes, rather than test cases you know will pass, is a skill in itself.

If you finish early

```
text = input("Text: ").lower()

for letter in "abcdefghijklmnopqrstuvwxyz":
    n = text.count(letter)
    if n > 0:
        print(f"{letter}: {n}")
```

It reads the whole text twenty-six times — once per `count()` call. For a paragraph nobody cares. For a book it is noticeably slow, and for a million books it is hopeless.

The alternative walks the text *once*, keeping a tally as it goes. That needs somewhere to store twenty-six running counts under twenty-six names, which is exactly what a dictionary is: lecture 7.